UNIVERSITY OF PISA

DEPARTMANT OF COMPUTER SCIENCE

Master programme in Data Science and Business Informatics

DATA MINING: FUNDAMENTAL

Project report: IMDb Dataset

Teachers:
Dino Pedreschi
Riccardo Guidotti
Andrea Fedele

Group:
Phuong Chi Huynh
Thi Hien Do
Narangoo Altangerel

Academic Year 2024/2025

Contents

Data Understanding and Preparation

1.1 Data semantics

The *IMDb Dataset* was recorded in a flat-file format with 16,431 rows and 23 columns (features). Each object represents a unique show or movie, described by a fixed set of attributes. To gain general knowledge about each feature of the dataset, we provide a semantic description including the meaning and data types, as shown below:

Variable Name	**Description**	**Source datatype**	**Measurement scale types**
`originalTitle`	The original title of the movie or show, in the original language.	`object`	Text
`rating`	IMDb rating class (e.g., [6, 7], [8, 9]).	`object`	Categorical
`startYear`	Represents the release year of a title. In the case of TV Series, it is the series start year.	`int64`	Numeric
`endYear`	TV Series end year. \N for all other title types.	`int64`	Numeric
`runtimeMinutes`	Primary runtime of the title, in minutes.	`int64`	Numeric
`awardWins`	Total number of awards the title has won.	`int64`	Numeric
`numVotes`	Number of votes the title received from users.	`int64`	Numeric
`worstRating`	The lowest possible rating (usually 1).	`int64`	Numeric
`bestRating`	The highest possible rating (usually 10).	`int64`	Numeric
`totalImages`	Total number of images for the title within the IMDb title page.	`int64`	Numeric
`totalVideos`	Total number of videos for the title within the IMDb title page.	`int64`	Numeric
`totalCredits`	Total number of credits (e.g., cast and crew) associated with the title.	`int64`	Numeric
`criticReviewsTotal`	Total number of critic reviews.	`int64`	Numeric
`titleType`	Type/format of title (e.g., movie, short, tvSeries, tvEpisode, video, etc.).	`object`	Text/Categorical
`awardNominationsExcludeWins`	Total number of nominations excluding awards won.	`int64`	Numeric
`canHaveEpisodes`	Indicates whether the title can have episodes (TRUE/FALSE).	`bool`	Categorical
`isRatable`	Whether the title can be rated by users (TRUE/FALSE).	`bool`	Categorical
`isAdult`	Whether the title is adult content (0=non-adult, 1=adult).	`int64`	Categorical
`numRegions`	Number of regions where this version of the title is available.	`int64`	Numeric
`userReviewsTotal`	Total number of user reviews the title received.	`int64`	Numeric
`ratingCount`	Total number of user ratings submitted for the title.	`int64`	Numeric
`countryOfOrigin`	Country where the title was primarily produced.	`object`	Categorical

We observed some inconsistencies in data types, which could affect analysis and modeling:

- `endYear` and `runtimeMinutes` are stored as `object`, potentially due to non-standard missing values.
- `awardWins` should be stored as `integer`.
- `isAdult` should be stored as `boolean`.
- `rating` represents a range, which may complicate calculations in subsequent data mining tasks.

Before exploring the distribution of variables and their statistics, we cleaned the data as follows:

- We replaced \N with NA and converted the necessary variables to their appropriate data types.
- To address the issue with calculations, we used the mean value of the rating range.

1.2 Distribution of variables and statistics

1.2.1 Summary of statistics

The dataset includes a wide array of attributes that describe different aspects of movies and shows. These summary tables 1.2 provides an overview of the primary statistical characteristics for each attribute and helps us make decision to remove what we thought we unnecessary variables, such as `bestRating`, `worstRating` and `isRatable` as each of these three variables had constant values across all data.

Variable	Count	Mean	Std	Min	25%	50%	75%	Max
`startYear`	16431.0	1991.87	26.12	1878.0	1978.0	1997.0	2013.0	2024.0
`endYear`	814.0	2001.57	18.45	1945.0	1989.0	2005.5	2018.0	2025.0
`runtimeMinutes`	11579.0	61.22	52.11	0.0	25.0	58.0	90.0	3000.0
`awardWins`	13813.0	0.49	2.97	0.0	0.0	0.0	0.0	145.0
`numVotes`	16431.0	1492.15	20137.71	5.0	15.0	36.0	148.5	966565.0
`worstRating`	16431.0	1.0	0.0	1.0	1.0	1.0	1.0	1.0
`bestRating`	16431.0	10.0	0.0	10.0	10.0	10.0	10.0	10.0
`totalImages`	16431.0	11.48	74.25	0.0	1.0	1.0	6.0	3504.0
`totalVideos`	16431.0	0.27	3.12	0.0	0.0	0.0	0.0	258.0
`totalCredits`	16431.0	61.35	174.02	0.0	16.0	34.0	65.0	15742.0
`criticReviewsTotal`	16431.0	2.79	15.41	0.0	0.0	0.0	1.0	533.0
`awardNominationsExcludeWins`	16431.0	0.56	3.96	0.0	0.0	0.0	0.0	197.0
`numRegions`	16431.0	3.55	5.85	1.0	1.0	1.0	3.0	69.0
`userReviewsTotal`	16431.0	7.23	66.50	0.0	0.0	0.0	2.0	5727.0
`ratingCount`	16431.0	1492.92	20145.39	5.0	15.0	36.0	149.0	967042.0

Table 1.2: Summary of statistics for numeric variables.

1.2.2 Distribution via histograms

Histograms allow us to examine the frequency of different values and identify any patterns, trends, or anomalies within the data. Below is an analysis of the most relevant variables (Fig. 1.1):

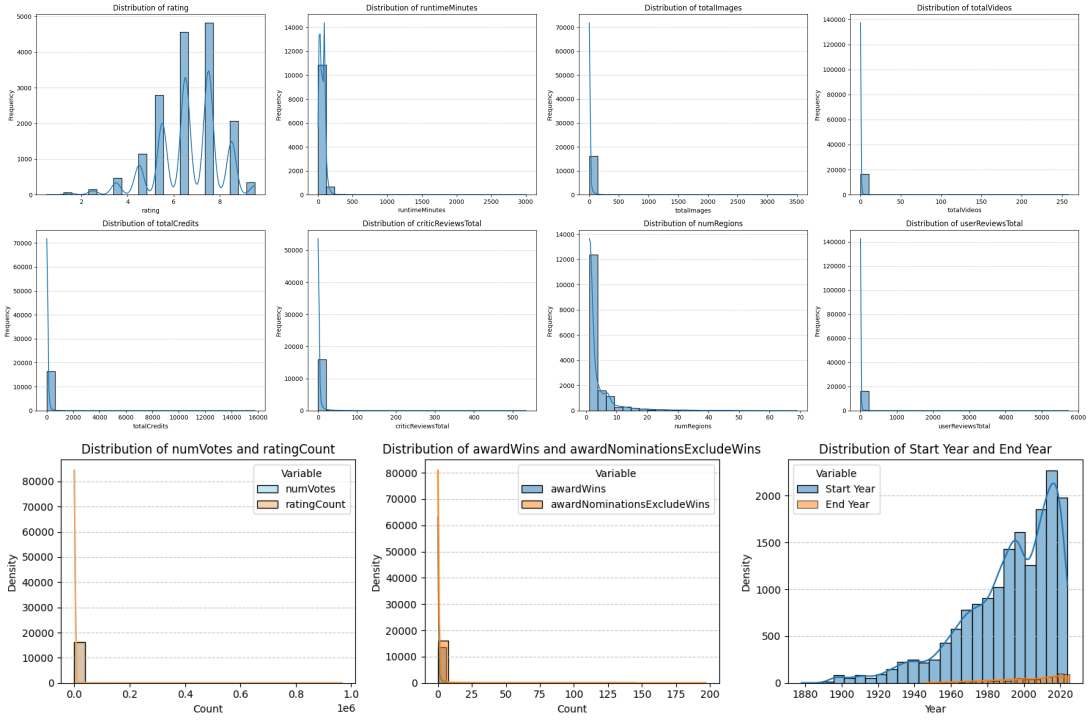


Figure 1.1: Histogram

Overall, the histogram analysis reveals a high degree of skewness across multiple variables in the dataset.

Most titles have mid-to-high ratings, typically between 6.5 and 7.5, with runtimes under 200 minutes and limited cast/crew credits and images. Engagement metrics, such as user and critic reviews, votes, and rating counts, are highly skewed, indicating a few dominant titles. Accessibility is mostly limited to a few regions, with global distribution being rare. Awards are minimal for most titles, and content production has surged significantly since the 1980s, especially in recent years.

1.2.3 Distribution via pie chart and bar chart

To gain insights into the categorical distributions within the IMDb dataset, we employed pie charts and bar charts (Fig. 1.2) to visualize the proportions of key attributes. The pie charts illustrate the distribution of attributes such as `titleType`, showing the relative prevalence of movies, TV episodes, and other content types, along with the distribution of `canHaveEpisodes` and `isAdult`. The majority of content is non-adult, reflecting a general audience focus, and most titles are standalone, with only a small proportion allowing multiple episodes. Meanwhile, the bar chart for `countryOfOrigin` displays the concentration of content originating from various countries, with a significant portion produced in the United States, followed by the United Kingdom, Japan, and other nations. The genres bar chart showcases the thematic diversity within the dataset, with “drama” and “comedy” as the leading genres, followed by widely represented categories like "short," "action," “documentary,” and “crime.” This broad genre distribution underscores the variety of entertainment interests within the dataset, catering to diverse audience preferences.

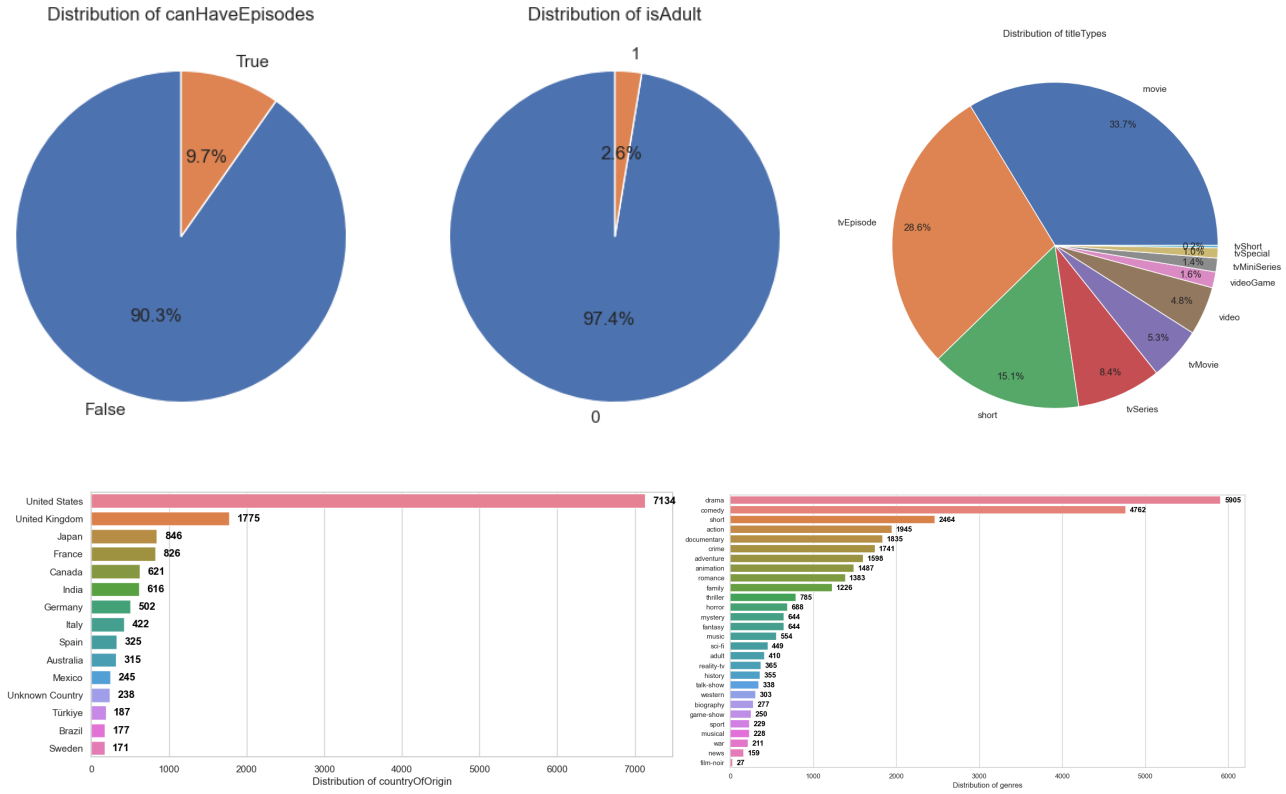


Figure 1.2: Pie chart and Bar chart

1.3 Assessment of data quality

1.3.1 Missing value handling

The dataset contains missing values in several columns as we can see on Figure 1.3 and detailed information in Table1.3. The strategies we employed to handle missing values were tailored to each attribute:

- `endYear`: Due to the very high percentage of missing values (95%), we decided to drop this feature from the dataset as it was deemed unreliable for meaningful analysis.
- `awardWins`: Missing values were filled with **0**, as it reflects that many titles did not win any awards.
- `runtimeMinutes`: We filled missing values using the **median** value of runtime for each `titleType`. This method ensures that the imputed values reflect the typical runtime for each content type.
- `genres`: Missing values in the `genres` column are filled with the **mode** of genres for each corresponding `titleType`. This approach preserves the categorical integrity of the dataset by inputting the most frequently occurring genre for each type of title.

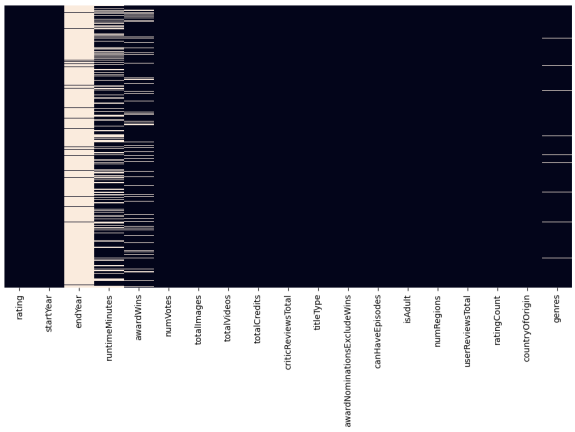


Figure 1.3: Missing value Heatmap

	No of missings	% of missings
`endYear`	15617	95.0
`runtimeMinutes`	4852	30.0
`awardWins`	2618	16.0
`genres`	382	2.0

Table 1.3: Summary of missing values.

1.3.2 Outlier and noise detection

We evaluated the presence of outliers in numerical attributes using statistical and visualization techniques such as boxplots. Each boxplot shows the distribution of numerical columns, including the median, quartiles, and potential outliers. The points beyond whiskers in the boxplots indicate the presence of outliers in various numerical attributes.

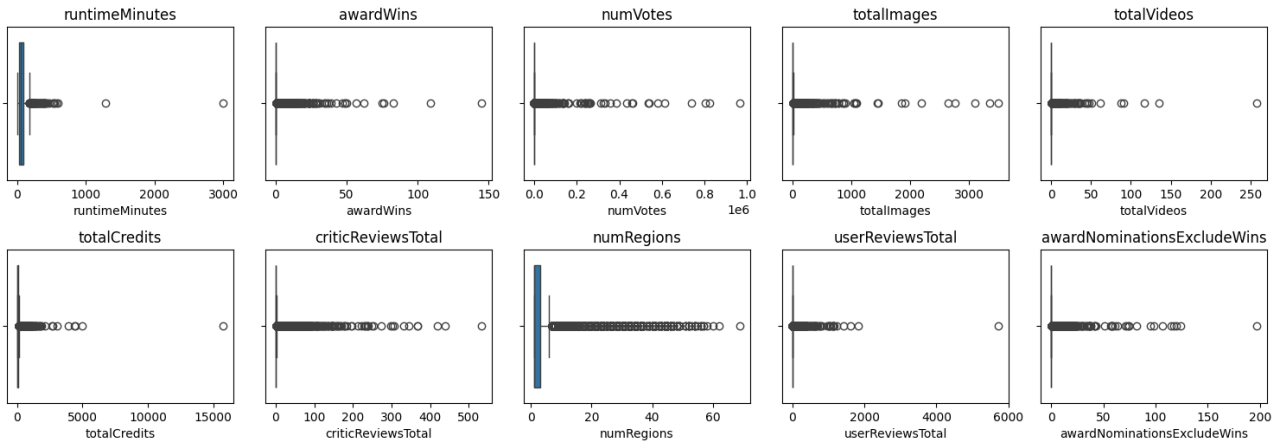


Figure 1.4: Boxplot

As observed in the boxplots (Fig. 1.4), many features in the dataset exhibit a significant number of outliers. Upon closer examination, we identified some extreme outliers that are excessively far from the majority of the data distribution and distant from other data points. These extreme values can distort key statistical measures, such as the mean, and negatively impact downstream analyses like clustering, classification, and regression. However, for pattern mining, since we employed manual binning techniques, the presence of these outliers had minimal impact on the results. Therefore, we retained them for this specific task.

To minimize their negative influence, we identified these extreme points as noise and decided to remove them from the dataset. Specifically, data points with: `runtimeMinutes` exceeding 1000, `awardWins` surpassing 100, `awardNominationsExcludeWins` exceeding 150, `totalCredits` above 6000, `userReviewsTotal` surpassing 2000, `totalVideos` exceeding 150 were excluded from the dataset.

Additionally, through statistical analysis, we also identified data points with `runtimeMinutes` = 0 as noise, since such values are unrealistic and do not correspond to valid runtime durations for movies or TV shows. Therefore, these data points were also excluded from the dataset.

All other outliers, which still provide meaningful information despite their deviation, were retained in the dataset to preserve its integrity and ensure our analysis remains comprehensive and reflective of real-world dynamics.

1.3.3 Variables Transformation

We observed that there are significant differences in scales between features. To address these inconsistencies and ensure uniformity in feature scales, we decided to apply **Z-Score Standardization method** to all numerical attributes. First, because Z-Score normalization ensures all features are transformed to a similar scale, preventing attributes with larger ranges, such as `runtimeMinutes` or `numVotes`, from disproportionately influencing our analysis. Second, outliers, as highlighted in section 1.3.2, are prevalent across many features of dataset. By scaling based on standard deviation, Z-Score normalization reduces the impact of these extreme values, ensuring they do not dominate the results.

1.3.4 Pairwise Correlations and Variable Elimination

To investigate the relationships among numerical features in the dataset, pairwise correlations were analyzed using the Spearman method (Fig. 1.5). We selected this method over Pearson's method after we visualized pairwise scatter plot matrix between attributes, we observed that many pairs of attributes have very weak linear relationships, which make Spearman's correlation a better choice over Pearson's method. Additionally, Spearman correlation is more robust to outliers which are present in many features in dataset.

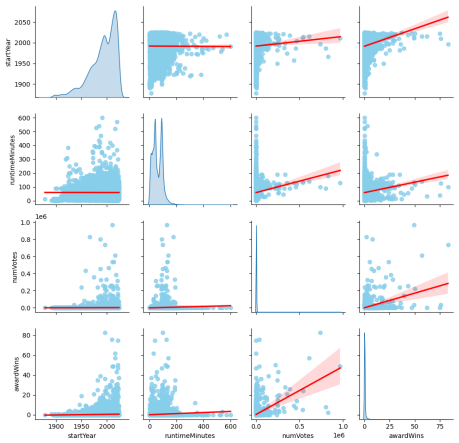


Figure 1.5: Pairwise regression plot matrix

The heatmap (Fig. 1.6) displays the Spearman correlation coefficients among numerical variables in the dataset, with values ranging from -1 to 1. Key observations include a strong positive correlation between `numVotes` and `userReviewsTotal` (0.66) which suggests that items with higher votes also tend to receive more user reviews. Similarly, `criticReviewsTotal` has a notable positive correlation with `numVotes` (0.56) and `userReviewsTotal` (0.55), indicating that higher critical reviews align with more user engagement and votes. On the other hand, `runtimeMinutes` and `startYear` have weak or negative correlations with most variables, showing minimal relationship with other features. We also observed that `ratingCount` has an extremely high correlation with `numVotes` (1.0), which mean both features provide same information making it redundant, so we decided to remove `ratingCount` from dataset.

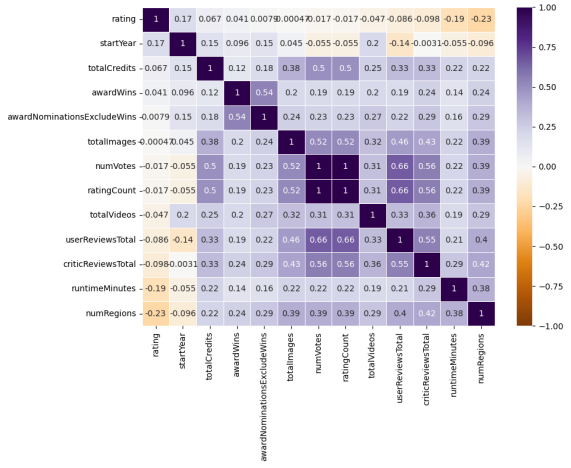


Figure 1.6: Correlation matrix

Clustering

In this section, we analyze three clustering techniques:

- **Centroid-Based Clustering** which focuses on partitioning data into clusters using distance-based centroids with K-Means.
- **Density-Based Clustering** which groups data points based on density regions with DBSCAN.
- **Hierarchical Clustering** builds a tree-like hierarchy of clusters through agglomerative.

We will evaluate each method based on parameter optimization, cluster quality, and interpretability and compare their effectiveness in revealing meaningful patterns in the dataset. To preprocess data for clustering, we excluded all categorical and boolean features from the dataset since clustering algorithms operate based on distance metrics which are meaningful only for numerical data. We also designed two separate scenarios to evaluate the impact of feature selection and dimensionality reduction on clustering performance:

Full features: We selected all the following **12 numerical** features for clustering: `rating`, `startYear`, `runtimeMinutes`, `awardWins`, `numVotes`, `totalImages`, `totalVideos`, `totalCredits`, `criticReviewsTotal`, `awardNominationsExcludeWins`, `numRegions`, `userReviewsTotal`.

Selected features: In the second scenario, we applied Principal Component Analysis (PCA) to reduce dimensionality and focus on the most informative features. We dropped 6 features: `rating`, `startYear`, `runtimeMinutes`, `totalImages`, `totalVideos`, `totalCredits` as they were found to contribute less variance to the clustering process based on PCA results. The remaining **6 numerical features** are: `awardWins`, `numVotes`, `criticReviewsTotal`, `awardNominationsExcludeWins`, `numRegions`, `userReviewsTotal`. As illustrated in the PCA graph (Fig. 2.1), the first 6 principal components explain 78.9% of the dataset's total variance; therefore, most of the dataset's information is still preserved despite reducing the number of features by half.

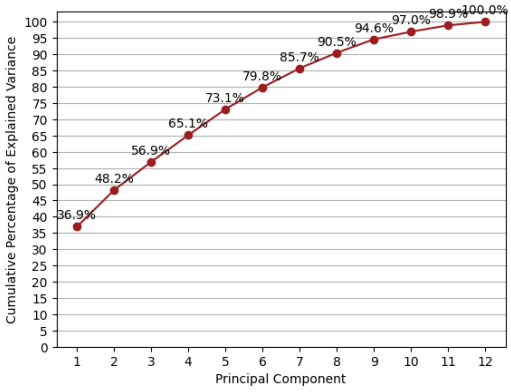


Figure 2.1: Cumulative Explained Variance Plot

2.1 Centroid-based

K-Means is an iterative clustering algorithm that partitions data into K distinct clusters, each represented by a centroid. The algorithm aims to minimize intra-cluster variance, ensuring data points within a cluster are close to their centroid. Selecting the **optimal K** is crucial for meaningful clustering. We use two key techniques:

- **The Elbow Method** evaluates the Within-Cluster Sum of Squares (SSE) for different values of K
- **The Silhouette Method** evaluates the Silhouette Coefficient, which measures how well-separated each cluster is from others. A higher score suggests better-defined clusters.

Full features

We tuned K from 2 to 20 and analyzed the clustering results using both the Elbow Method and the Silhouette Method (Fig. 2.2).

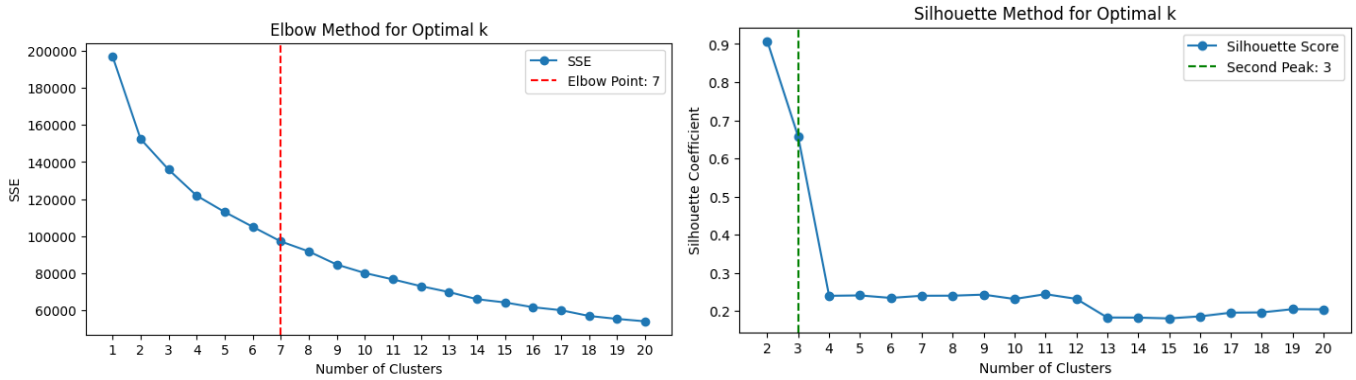


Figure 2.2: Elbow Method and Silhouette for Optimal K

We observed that the steep drop in the Silhouette Score after K = 3 indicates a significant reduction in clustering quality when increasing the number of clusters to K = 4 or beyond. With K = 3, the clustering results produced three clusters with a Silhouette Score of 0.68, distributed as cluster 0: 563 points, cluster 1: 43 points, and cluster 2: 15,817 points. This distribution is imbalanced in terms of cluster sizes, with one dominant cluster and two significantly smaller clusters. However, the higher Silhouette Score of 0.68 indicates that the clusters remain well-defined and cohesive. Therefore, based on the higher Silhouette Score, the clear peak at K = 3, we selected K = 3 as the optimal number of clusters for our IMDb dataset.

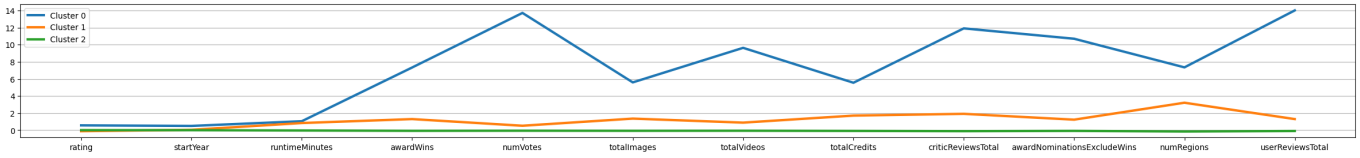


Figure 2.3: Parallel chart for full features

We can try to interpret the clusters generated by the analysis (Fig. 2.3):

- Cluster 0 (563 points): Titles with significantly higher values across all features, characterized by extensive audience engagement, numerous awards, and critical acclaim.
- Cluster 1 (43 points): Titles with moderately higher values across features, suggesting moderate engagement and recognition in terms of votes, reviews, and awards.
- Cluster 2 (15,817 points): Titles with low average values across all features, representing low-profile or less popular titles with minimal awards, votes, and reviews.

Selected features

Following the same process as in Scenario 1, we tuned K from 2 to 20 and yielded two potential optimal values for K with K = 5 from the Elbow Method, K = 3 from the Silhouette Method. To make an informed decision, we carefully inspected values in the range of 3 to 5, as shown in the table 2.1 below:

Metric	**K = 5**	**K = 4**	**K = 3**
SSE	32388.94	38912.02	46856.88
Silhouette Score	0.79	0.8	0.86
Cluster Sizes	[964, 73, 18, 66, 15302]	[1024, 103, 21, 15275]	[15827, 561, 35]

Table 2.1: Comparison of clustering metrics for different values of K

While K = 3 achieved the highest Silhouette Score (0.86), the resulting cluster distribution was imbalanced. On the other hand, K = 5 resulted in a fragmented distribution, breaking down clusters from K = 4 into smaller, less interpretable groups. After evaluating both the Silhouette Score and cluster distribution, we selected K = 4 as the optimal choice since K= 4 produced clusters with more reasonable distribution while maintaining a good Silhouette Score (0.80).

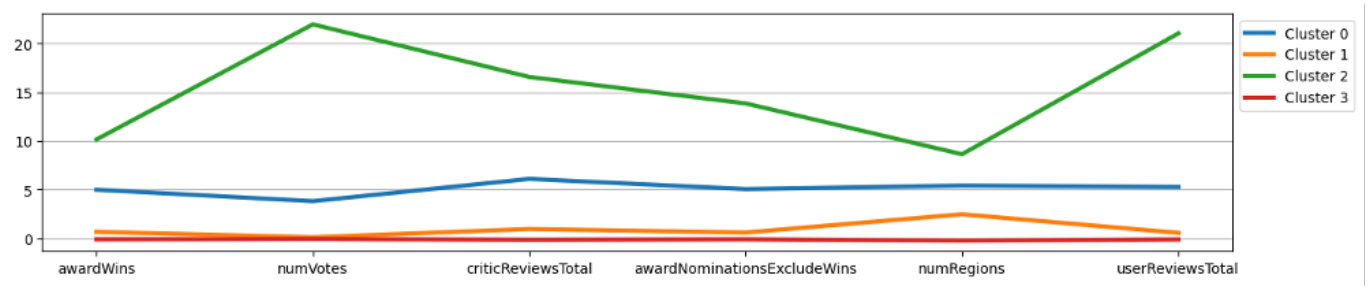


Figure 2.4: Parallel chart for selected features

This plot has a very similar pattern and interpretation like in the first case but focuses on fewer and highly relevant numerical features.

- Cluster 0 (1,024 points) represents titles with moderately high values across key features, indicating reasonably popular content with slight peaks in `criticReviewsTotal` and `numRegions`.
- Cluster 1 (103 points) includes titles with lower overall engagement but moderate visibility in specific aspects, such as regional availability.
- Cluster 2 (21 points) stands out with exceptionally high values across all key features, with pronounced peaks in `numVotes` and `userReviewsTotal`.
- Cluster 3 (15,275 points) consistently remains at minimal values across all displayed features.

Evaluation

In Scenario 1, using 12 original numerical features, the clustering provided richer feature-level insights but suffered from higher SSE and weaker clustering quality. In Scenario 2, using 6 PCA-reduced numerical features, the clustering achieved a lower SSE, a higher Silhouette Score, and a more balanced cluster distribution. As a result, we think scenario 2 outperformed Scenario 1 and delivered better clustering quality and better overall results, making it the preferred choice for our dataset.

2.2 Density-based

DBSCAN algorithm works by identifying dense regions of data points in the dataset based on two key parameters:

- Epsilon (ϵ): Defines the maximum distance between two points for one to be considered part of the other's neighborhood.
- MinPts: Specifies the minimum number of points required to form a dense region (core point).

Finding optimal values for ϵ (epsilon) and MinPts is crucial for DBSCAN as it directly influences cluster formation and noise identification. To determine the best value for ϵ, we analyzed the k-th nearest neighbor (k-NN) distances for each data point, plotting these distances at varying k-values from 2nd, 3rd, 5th, 10th, 15th, 20th to 50th neighbors as shown in (Figure 2.5). The plot shows an elbow point where the curve shifts from a gradual slope to a sharp increase indicating the optimal ϵ value, determined using the Knee Locator

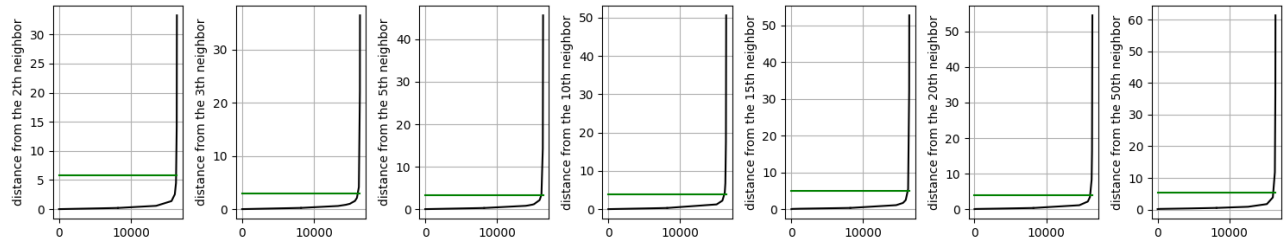


Figure 2.5: k-nearest neighbors' distance of full dataset

Table 2.2: k-NN and ε of full set

	2	**3**	**5**	**10**	**15**	**20**	**50**
ε	5.86	2.93	3.78	4.89	5.71	4.04	4.18

Table 2.3: k-NN and ε of selected features set

	2	**3**	**5**	**10**	**15**	**20**	**50**
ε	3.33	2.2	2.34	3.36	2.36	2.67	5.73

In our DBSCAN analysis, we experimented with various parameter combinations for both the full features set and the selected features set (Table 2.2 and 2.3). The results revealed a high degree of cluster imbalance in both cases. Additionally, we observed that higher MinPts values consistently led to a single dominant cluster with several noise points, failing to capture meaningful patterns or sub-groups in the data.

For the **full feature set** using MinPts = 3 and $\epsilon = 2.93$, the algorithm produced seven clusters with the distribution: [194, 16,211, 3, 4, 5, 3, 3]. A single dominant cluster (16,211 points) encompassed the vast majority of data points, while the remaining 5 clusters contained only a handful of points each, ranging from 3 to 5 data points and 194 noise points. This imbalance resulted in a Silhouette Score of 0.65, suggesting moderate clustering quality. However, in our opinion we think that the score is heavily influenced by the dominant cluster and does not accurately represent the fragmented and poorly separated smaller clusters. When we increased MinPts, the clustering consistently collapsed into one large cluster with many noise points, further reducing the interpretability of the results.

For the **selected features set** with MinPts = 3 and $\epsilon = 2.2$, DBSCAN generated four clusters with the distribution: [124, 16,293, 3, 3]. Similar to the full feature set, one dominant cluster (16,293 points) captured nearly all the data, while two other clusters contained just 3 points each, and 124 points were classified as noise. Despite this extreme imbalance, the Silhouette Score was exceptionally high at 0.9. However, like the first case we think that this high score is misleading, as it reflects the tight cohesion of the dominant cluster while overlooking the poor separation and representation of the smaller clusters. Increasing MinPts in the selected features set also resulted in the creation of a single large cluster with several noise points, mirroring the pattern observed in the full feature set.

2.3 Hierarchical

In this section, we used an agglomerative (bottom-up) clustering algorithm, experimenting with Ward, Group Average (Average), Max (Complete), Min (Single) linkage methods and the distance metric of Euclidean. To compare Silhouette scores and analyze cluster structures, we used the optimal k values found through k-means clustering (k = 3 for the full dataset and k = 4 for the cut dataset). We also tested multiple k values and observed no significant difference in results.

Table 2.4: Full dataset

Linkage	**Cluster sizes**	**Silhouette score**
Ward	[15577, 65, 781]	0.5896
Average	[12, 16409, 2]	0.9412
Max	[53, 16364, 6]	0.9042
Min	[16421, 1, 1]	0.9435

Table 2.5: Feature-selected dataset

Linkage	**Cluster sizes**	**Silhouette score**
Ward	[141, 649, 13, 15620]	0.8257
Average	[12, 16404, 3, 4]	0.9692
Max	[35, 4, 16381, 3]	0.9544
Min	[16420, 1, 1, 1]	0.9779

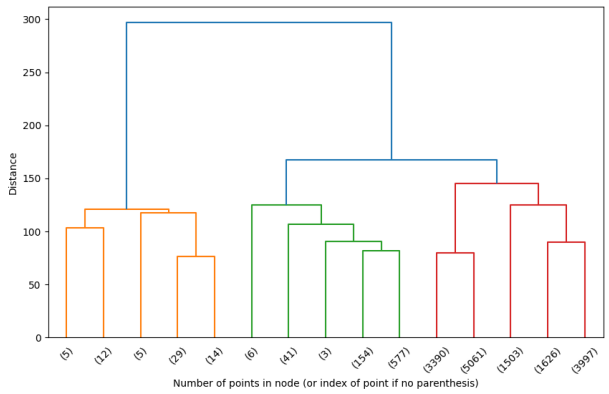


Figure 2.6: Dendrogram of full dataset

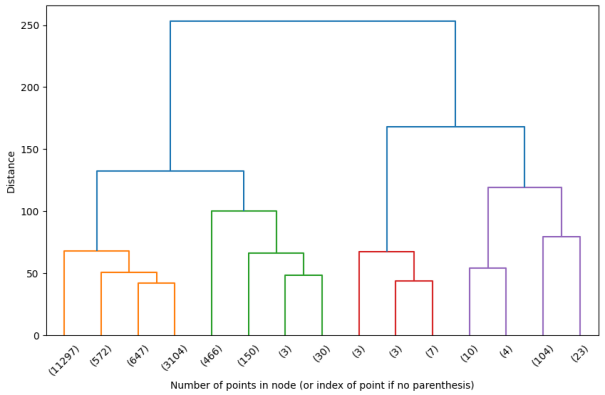


Figure 2.7: Dendrogram of feature-selected dataset

For the **full dataset**: the Ward method produces a relatively balanced clustering, with one large cluster (15577 data points) and two smaller ones. A Silhouette score of 0.5896 indicates a reasonably good but not perfect clustering, suggesting potential overlap between clusters. The two other methods, Complete and Average, mostly concentrate on a single, excessively large cluster and two very small, insignificant clusters, despite having much higher Silhouette scores than Ward. The Single method is completely unsuitable as it creates only one large cluster and almost ignores all other potential clusters.

For the **feature-selected dataset**: The results are quite similar to the full dataset, with the Ward method being the best option, but with a significantly improved Silhouette score (0.8257) compared to the full dataset.

2.4 Final discussion

During the analysis, K-Means was the only method capable of partitioning the clusters in a balanced manner and achieving a good Silhouette score when applied to the feature-selected dataset. In contrast, DBSCAN could not provide optimal clustering results despite trying various values of eps and minPts, as most results only produced large clusters covering almost the entire dataset and some noise points. Hierarchical methods, including Ward's Method, also created unbalanced clusters and demonstrated that the sparse data structure and complexity of the dataset made clustering less effective.

Classification

This part of the analysis, we selected `titleType` as the target variable. Our objective is to build a model capable of predicting 10 different classes of `titleType`. Before applying the classification algorithms, we performed several preprocessing steps for both the training and test datasets to ensure data consistency and quality. The test set was carefully prepared to align with the training set, undergoing the same preprocessing steps, including missing value handling, attribute selection, and normalization of numerical features. To handle categorical features for both training set and test set, the target variable titleType and the feature `rating` were label-encoded to convert categorical labels into numerical values. The genres feature, initially containing multiple genre combinations, was grouped into 15 broader categories to simplify analysis, followed by One-Hot Encoding to create binary features for each genre group. Similarly, the countryOfOrigin feature was processed by identifying the top 10 most frequent countries and grouping less frequent countries under 'Other Countries'. After grouping, One-Hot Encoding was applied to create binary features for each country category. After preprocessing, our training dataset consisted of 16,423 rows and 41 columns, while the test dataset contains 5,478 rows and 41 columns ready for model training and evaluation.

In the upcoming analysis, we will implement and compare three classification algorithms: K-Nearest Neighbors (KNN), Naive Bayes, and Decision Trees. KNN will use normalized data, as it's a distance-based algorithm sensitive to feature scale differences. In contrast, Naive Bayes, which relies on probability distributions, and Decision Trees, which split data based on feature thresholds, do not require normalization. The performance of these models will be assessed using key evaluation metrics, including accuracy, precision, recall, F1-score, and ROC curves.

3.1 KNN

To optimize the performance of the KNN model, we performed a Grid search combined with cross-validation across a range of 1 to 50 neighbors to search for the best hyperparameters. The best configuration identified was as `'metric': 'cityblock', 'n_neighbors': 8, 'weights': 'distance'`.

The evaluation table (Table 3.1) below summarizes the Precision, Recall, and F1-Scores across the 10 classes:

Class	titleType	Precision	Recall	F1 score
0	tvShort	1.00	0.06	0.12
1	tvSpecial	0.54	0.43	0.48
2	tvMiniSeries	0.59	0.28	0.38
3	videogame	0.79	0.67	0.72
4	video	0.66	0.53	0.59
5	tvMovie	0.46	0.28	0.35
6	tvSeries	0.88	0.92	0.90
7	short	0.93	0.93	0.93
8	tvEpisode	0.86	0.95	0.90
9	movie	0.87	0.89	0.88

Table 3.1: Precision, Recall, and F1 Score for Different titleType Classes.

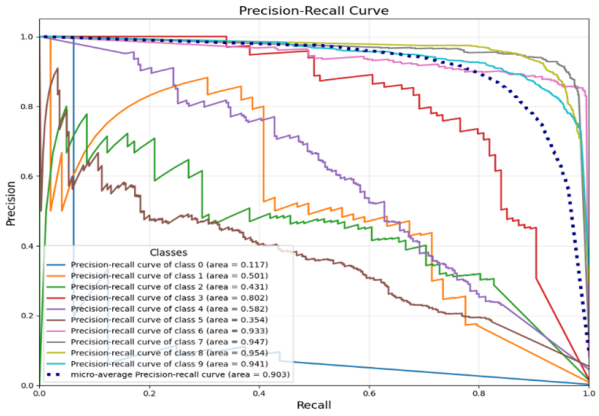


Figure 3.1: Precision – Recall Curve.

The overall accuracy of 85% indicates that the KNN model performed well, correctly classifying classes across the dataset.

To gain more insights, we combined the Precision-Recall curve (Fig. 3.1) and the evaluation table (Table 3.1) to analyze both the strengths and weaknesses of the classification model. High-performing classes, such as 6, 7, 8, and 9 consistently achieve high Precision, Recall, and F1-Scores, supported by their well-positioned PR curves in the upper-right quadrant, indicating an excellent balance between precision and recall. In contrast, classes 0, 1, 2 and 5 display significant weaknesses, characterized by poor recall scores and low F1-Scores despite occasional high precision, as exemplified by Class 0, where precision is 1.00 but recall is only 0.06. These shortcomings are mirrored in their PR curves clustering near the lower-left corner, highlighting their inability to effectively identify true positives. Mid-performing classes, such as 3 and 4, demonstrate moderate results. The micro-average Precision-Recall curve (AUC = 0.903) indicates strong overall model performance; however, the underperforming classes have a measurable negative impact on the global results.

We further analyzed the confusion matrix (Figure 3.2) to understand the patterns of correct predictions and misclassifications across the classes. The diagonal values represent accurate predictions for each class, while off-diagonal values indicate misclassifications. Higher values along the diagonal suggest good performance for specific classes, such as Class 6, Class 7, Class 8, and Class 9. These classes exhibit low misclassification percentages (see Table 3.2), ranging from 4.63% to 11.13%, highlighting the model's strong predictive performance in these categories. In contrast, Class 0 demonstrates the highest misclassification rate 93.75%, with only 1 correctly predicted sample out of 16 total samples. Many of these misclassifications are distributed across Class 7 and Class 8. Similarly, Class 5 suffers from a 71.91% misclassification rate, with numerous samples being incorrectly predicted as Class 8 and Class 9. Class 2 also displays high misclassification percentages, frequently being misclassified into Class 6. Mid-performing classes such as Class 3 and Class 4 exhibit moderate misclassification rates of 32.98% and 46.8%, respectively, with misclassifications primarily directed towards Class 8 and Class 9.

Class	Misclassification %
0	93.75
1	57.14
2	71.60
3	32.98
4	46.8
5	71.9
6	8.05
7	7.05
8	4.62
9	11.13

Table 3.2: Misclassification Percentage by Class

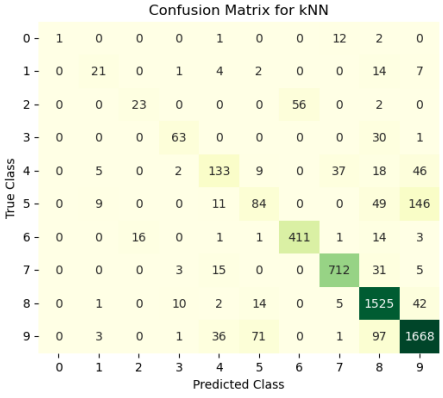


Figure 3.2: Confusion matrix

We also analyzed the ROC Curve (Fig. 3.5) to evaluate the model's ability to distinguish between different classes by examining the trade-off between the True Positive Rate (TPR) and the False Positive Rate (FPR) across all thresholds. The Area Under the Curve (AUC) serves as a critical performance indicator, with higher values indicating stronger classification performance. Classes such as 6, 7, 8, 9 exhibit exceptional

performance, with AUC values exceeding 0.97, reflecting the model's robust ability to correctly classify these categories. Moderately performing classes, including Class 1, 2, 3, 4, 5 display AUC values ranging from 0.85 to 0.95, indicating reasonably good predictive performance but with some room for improvement, particularly in reducing false positives. In contrast, Class 0 shows the lowest AUC at 0.71, highlighting significant difficulties in distinguishing this class from others, which aligns with the previously observed high misclassification rate. The micro-average AUC of 0.97 indicates strong overall model performance, while the macro-average AUC of 0.91 reflects a slight negative impact from poorly performing classes like Class 0. In conclusion, The KNN classification model demonstrates strong performance in predicting larger and well-represented classes such as Class 6(tvSeries), Class 7(short), Class 8 (tvEpisode), and Class 9 (movie). However, it struggles with smaller or underrepresented classes such as Class 0 (tvShort), Class 1(tvSpecial), class2 (tvMovie), and Class 5(tvMiniSeries), due to poor recall and high misclassification rates.

3.2 Naïve Bayes

This section evaluates the performance of two Naive Bayes classifiers -— Gaussian Naive Bayes (GNB) and Categorical Naïve Bayes (CategoricalNB) -— on a dataset exhibiting significant skewness and class imbalance.

We begin with Gaussian Naive Bayes (GNB). This method assumes that features follow a Gaussian (normal) distribution. GNB achieved an accuracy of 0.46, considerably lower than k-Nearest Neighbors (0.85). This low accuracy indicates GNB's struggle to distinguish between classes. The macro-averaged precision, recall, and F1-score were 0.42, 0.38, and 0.30, respectively, suggesting poor overall classification performance. Per-class metrics showed particularly low precision and recall for several classes (e.g., 0-3, 5, and 6), indicating difficulty in both accurate classification and capturing actual instances. Class 9 exhibited high precision (0.92) but low recall (0.18), indicating a tendency to confidently predict this class only when highly certain, missing many true instances. These results might from:**(1)** the violation of GNB's feature independence assumption, likely due to high feature correlation; **(2)** data skewness, violating the Gaussian distribution assumption and impacting probability estimations; and **(3)** class imbalance, biasing the model towards majority classes and hindering the identification of minority classes (similar to the results with kNN). Although weighted averaging attempts to mitigate the imbalance, it does not fully address the issues of skewed distributions and the violation of the feature independence assumption, thus significantly limiting GNB's performance.

Table 3.3: Performance Metrics for GaussianNB and CategoricalNB

Model	**Accuracy**	**Macro Avg F1**	**Weighted Avg F1**
GaussianNB	0.46	0.30	0.46
CategoricalNB	0.82	0.50	0.79

We also explored Categorical Naive Bayes (CategoricalNB), a method more suitable for datasets containing categorical features as ours.To utilize the data for this section, we discretized the numerical variables using `KBinsDiscretizer(n_bins=4, encode='ordinal', strategy='kmeans')`. CategoricalNB achieved a significantly higher accuracy of 0.82 compared to GNB's 0.46. This improvement in overall accuracy demonstrates the suitability of CategoricalNB for this dataset, likely due to the presence of categorical features, which addresses the limitations of GNB related to distributional assumptions. However, a closer look at the per-class metrics reveals that CategoricalNB shows strong performance on majority classes (6, 7, 8, and 9) but still struggles with minority classes (0, 1, and 5). The macro-averaged precision, recall and F1-score of 0.60, 0.48, and 0.50 respectively, while higher than GNB, demonstrate the model's difficulty with class imbalance. The weighted average F1-score (0.79) is much closer to the overall accuracy, which shows that much of the accuracy comes from the model's ability to predict the majority classes. This highlights the importance of selecting a classification algorithm appropriate for the characteristics of the data. While CategoricalNB is clearly a better choice than GNB, addressing the class imbalance is crucial for further performance improvement, particularly for the minority classes.

3.3 Decision Trees

Decision Trees are good at handling mixed data (both numbers and categories) and give a good overview of how the model classifies the data. When building our model, we used a method called Random Search to find the best settings (parameters). The best results we got were with `min_samples_split = 30, min_samples_leaf = 5, max_depth = 13` and `criterion = 'entropy'`, which gave us an accuracy of 0.8756 on the training data. When we tested it on the test data, the overall accuracy was 0.88, which means the model predicts pretty well. However, if we look closer, the performance is not the same for all the different classes. The results are similar to what we got with kNN, but a bit better in terms of overall accuracy. Classes 3, 6, 7, 8, and 9 had high accuracy and recall, while classes 0-2, 4, and 5 did not do as well. Class 0 was especially difficult to predict (both accuracy and recall were 0), just like we saw with the kNN model.

Class	Precision	Recall	F1-Score	Support
0	0.00	0.00	0.00	16
1	0.34	0.33	0.33	49
2	0.69	0.53	0.60	81
3	0.98	0.93	0.95	94
4	0.71	0.51	0.60	250
5	0.64	0.38	0.48	299
6	0.92	0.96	0.94	447
7	0.91	0.96	0.93	766
8	0.93	0.94	0.94	1599
9	0.87	0.95	0.90	1877

Table 3.4: Precision, Recall, F1-Score, and Support per Class with decision trees.

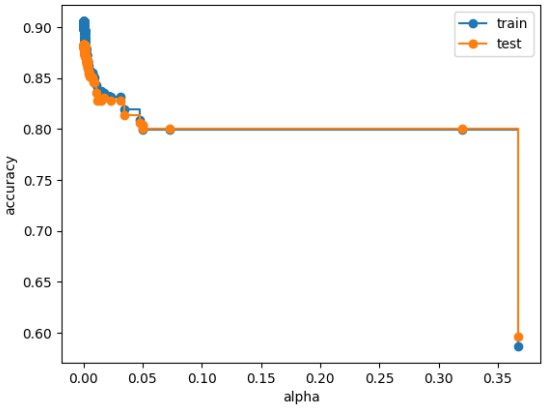


Figure 3.3: Accuracy vs Alpha plot.

To make the model easier to understand and to avoid overfitting (which is when the model learns the training data too well and does not work well on unseen data), we used a technique called pruning. We chose the best `ccp_alpha` value (0.02) and removed unnecessary branches (using `prune_duplicate_leaves`). Pruning simplifies the tree structure, making the model easier to understand and visualize, and it can also improve how well it works on unseen data by reducing variance.

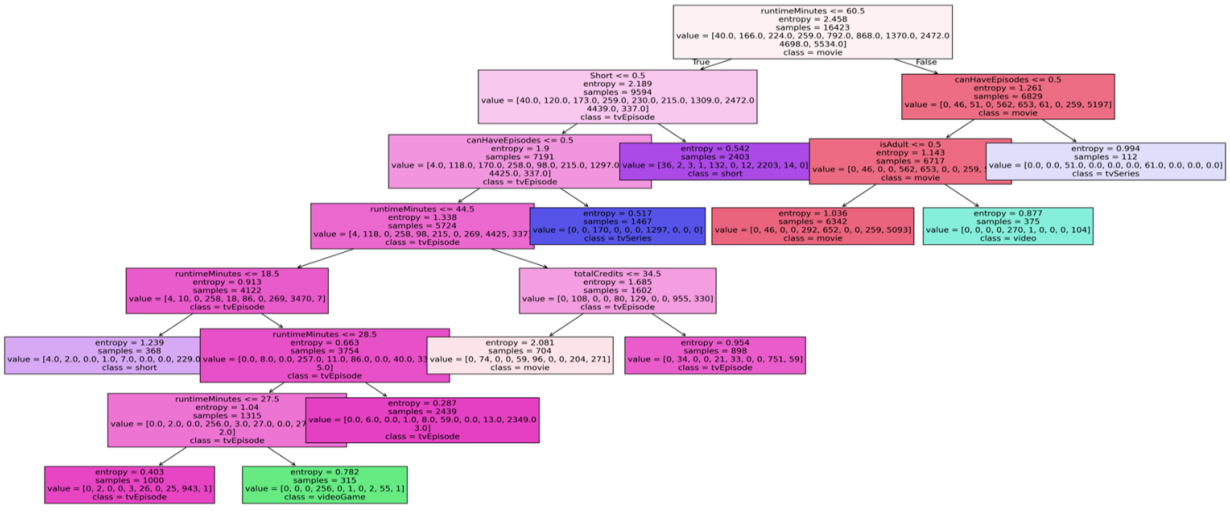


Figure 3.4: Decision Tree

The resulting pruned decision tree (Fig. 3.4) offers a clear and concise visualization of how the dataset is classified based on key features. At the root, the model initiates the classification by splitting the data on the feature `runtimeMinutes`, emphasizing that this is the most critical initial factor in determining class labels. Instances where `runtimeMinutes` is less than or equal to 60.5 are routed down branches leading to classes such as `tvEpisode` and `Short`, while instances with `runtimeMinutes` greater than 60.5 branch towards categories like `movie` and `tvSeries`. This primary split effectively partitions shorter content, such as TV episodes or short films, from feature-length productions.

Subsequent splits further refine the classification using features such as `canHaveEpisodes`, `Short`, and `totalCredits`. For example, the `canHaveEpisodes` feature differentiates episodic content (`tvSeries` and `tvEpisode`) from standalone productions (`movie`), providing a clear pathway for categorizing series-based content. The feature `totalCredits` is particularly useful in instances where runtime alone does not offer a definitive classification, helping to distinguish between productions with large versus small casts and crews, even when runtime similarities exist. For shorter content, splits on the `Short` feature help differentiate between very short films or specials and episodic TV content.

Entropy values at each node signify the level of uncertainty before each split, with lower entropy at leaf nodes indicating that the tree has successfully grouped the samples into more homogenous categories. Leaf nodes display the majority class as the predicted outcome, with the `value` parameter showing the distribution of samples across all target classes at that node.

3.4 Final Conclusion

The classification task aimed to predict the `titleType`, a categorical variable with 10 distinct classes, using a combination of categorical and numerical features. For this dataset, the Decision Tree emerged as the recommended model due to its consistent and reliable classification performance across all metrics.

Model	Macro-average F1-score	Macro-average AUC	Accuracy
kNN	0.62	0.91	0.85
GaussianNB	0.30	0.86	0.46
CategoricalNB	0.50	0.95	0.82
Decision Tree	0.67	0.90	0.88

Table 3.5: Model evaluation results

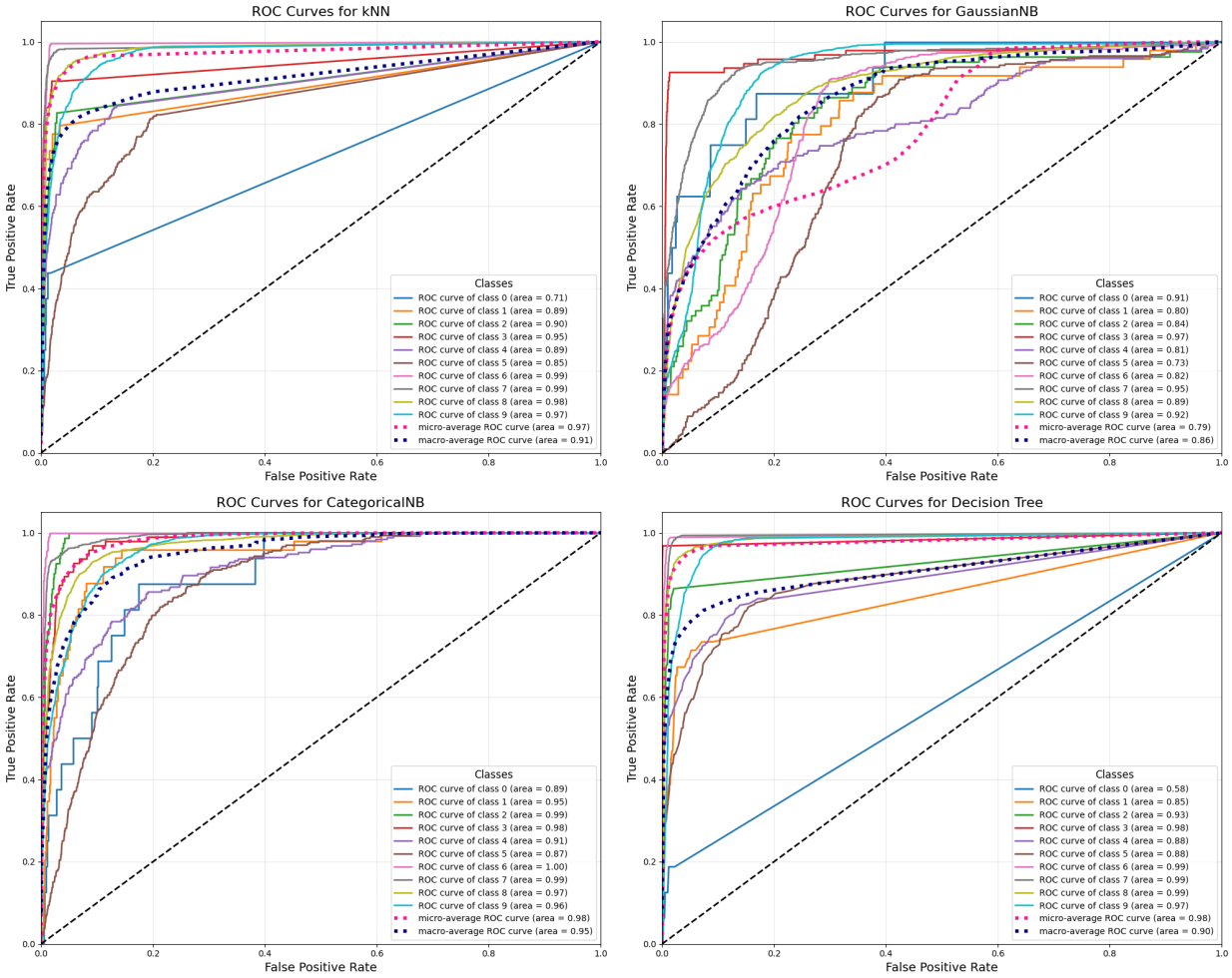


Figure 3.5: ROC Curves

Conclusion based on the metrics presented in the table 3.5 and observations from the ROC curves (Fig. 3.5:

- **kNN** demonstrated competitive performance, achieving strong results on balanced data, particularly for classes with dense distributions. However, it struggled with imbalanced classes, especially underrepresented ones like low-frequency title types. Compared to **Decision Tree** and **CategoricalNB**, kNN was less effective in handling such challenges.
- **GaussianNB** exhibited the lowest performance among the models, with significant difficulties in handling unbalanced classes and categorical features. GaussianNB was less effective than **CategoricalNB** and **Decision Tree**, resulting in high misclassification rates for complex or underrepresented classes.
- **CategoricalNB** showed notable improvement over GaussianNB, especially in processing categorical data like `genres` and `countryOfOrigin`. However, it was still less robust than the **Decision Tree** in managing interactions among multiple features and distinguishing closely related classes.
- **Decision Tree Classifier** delivered the most consistent results, outperforming other models in distinguishing between closely related classes. It effectively handled both categorical and numerical variables and achieved the highest accuracy and robustness across all classes.

In conclusion, while all models demonstrated unique strengths, the Decision Tree provided the best combination of interpretability, accuracy, and reliability, making it the most suitable choice for this dataset.

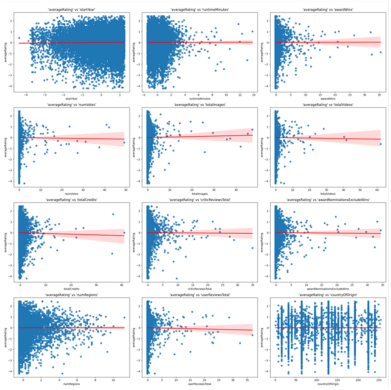
Regression

4.1 Univariate Regression

Simple Regression

To select variables for the Simple Regression analysis with the target variable averageRating, we used both **regression plots** and **correlation analysis** as our guiding tools:

- Linear relationship in the regression plot: Most of the regression lines (red lines) are almost horizontal, indicating that the linear relationship between `averageRating` and the variables is very weak. The large scatter plot of the data points around the regression line reflects the very low predictive ability of these variables.
- Correlation analysis: The correlation table shows that most of the correlation coefficients between `averageRating` and other variables are close to 0. This confirms that no variable has a strong linear relationship with `averageRating`.



	Variable	Correlation
0	averageRating	1.000000
1	awardNominationsExcludeWins	0.012961
2	startYear	0.011551
3	awardWins	0.010596
4	genres	0.008231
5	isAdult	0.007650
6	criticReviewsTotal	0.004565
7	canHaveEpisodes	0.002839
8	userReviewsTotal	0.002065
9	runtimeMinutes	0.001951
10	totalCredits	0.001356
11	numRegions	-0.000224
12	totalVideos	-0.001678
13	countryOfOrigin	-0.002131
14	numVotes	-0.003418
15	totalImages	-0.003799
16	titleType	-0.011185

Figure 4.1: Regression plot.

Based on these analyses, we selected the six variables with the relatively strongest correlations with `averageRating` for further modeling: `awardNominationsExcludeWins`, `startYear`, `awardWins`, `genres`, `isAdult`, `titleType`. For each of these variables, we ran five regression models: **Linear Regression, Lasso, Ridge, Decision Tree and kNN**.

Table 4.1: Simple Regression

Target	Predictors	Model	Coefficient Behavior	R^2	MSE	MAE
`averageRating`	`awardNominations ExcludeWins, startYear, awardWins, genres, isAdult, titleType`	Linear	Small coefficients (minimal variation)	≈ 0	1.832	1.062
		Ridge	Shrunk coefficients due to ℓ_2-regularization	≈ 0	1.832	1.062
		Lasso	All coefficients reduced to 0	≈ 0	1.832	1.062
		Decision Tree	Non-linear splits, depends on predictor	< 0	$1.833 - 1.843$	$1.062 - 1.067$

Target	Predictors	Model	Coefficient Behavior	R^2	MSE	MAE
		kNN	Non-linear relationships, heavily impacted by noise	< 0	$1.843 - 2.938$	$1.064 - 1.337$

Overall, none of the univariate models—linear or non-linear—demonstrated significant predictive value for averageRating. All three models—Linear Regression, Ridge, and Lasso—confirmed the lack of a meaningful relationship between averageRating and the independent variables. The R^2 values were consistently very low, while MSE and MAE showed no substantial variation across models. Lasso eliminated all predictors, reflecting their negligible contribution to the model, while Ridge and Linear Regression retained them, but with no improvement in performance. These results align with earlier findings of weak correlations and near-horizontal regression lines, underscoring the absence of strong linear relationships between the target variable and its predictors.

Turning to the non-linear models, the Decision Tree produced non-linear splits dependent on the predictor, yet the R^2 values remained mostly negative or near zero. The MSE (ranging from approximately 1.833 to 1.843) and MAE (approximately 1.062 to 1.067) showed no meaningful improvement over the linear models. The kNN model, while attempting to capture non-linear relationships, was significantly impacted by noise in the data. This resulted in generally negative R^2 values and wider ranges for both MSE (approximately 1.843 to 2.938) and MAE (approximately 1.064 to 1.337). This poor performance reflects both the lack of discernible structure in the data and the known sensitivity of kNN to noisy inputs.

Multiple regression

We also ran Multiple Regression with all six variables simultaneously as predictors for averageRating. For non-linear methods (Decision Tree and kNN), hyperparameter tuning was performed using GridSearchCV.

Table 4.2: Multiple Regression

Target	Predictors	Model	R^2	MSE	MAE
averageRating	awardNominationsExcludeWins, startYear, awardWins, genres, isAdult, titleType	Linear	≈ 0	1.832	1.061
		Ridge	≈ 0	1.832	1.061
		Lasso	≈ 0	1.832	1.062
		Decision Tree	0.001	1.830	1.062
		kNN	-0.058	1.938	1.093

Notes: Best parameters of Decision Tree: `{'max_depth': 3, 'min_samples_leaf': 2, 'min_samples_split': 5}`.
Best parameters of kNN: `{'metric': 'euclidean', 'n_neighbors': 15, 'weights': 'uniform'}`.

- Linear Models (Linear, Ridge, Lasso): The results were consistent with the univariate analysis, with $R^2 \approx 0$, MSE (≈1.832), and MAE (≈1.061-1.062), showing no improvement from combining predictors.
- Decision Tree: With tuned hyperparameter, the model achieved a slightly positive $R^2 = 0.001 \approx 0$, marginally improving over simple regression. However, the predictive value remained negligible, with MSE (≈1.830) and MAE (≈1.062).
- kNN: Despite tuning hyperparameter, kNN continued to underperform, with a negative $R^2 = -0.058$, MSE (≈1.938), and MAE (≈1.093), consistent with the noise-sensitive behavior observed in simple regression.

Both univariate and multivariate analyses consistently showed that the predictors, individually or collectively, failed to adequately explain the variability in `averageRating`. Even with non-linear methods and hyperparameter optimization, no substantial improvement was observed. This aligns with earlier findings of weak correlations and minimal predictive power, likely due to: (1) a lack of strong relationships between predictors and the target variable, and (2) noise and insufficient information within the predictors.

4.2 Multivariate

In multivariate regression analysis, we applied models with multiple target variables (`averageRating, runtimeMinutes`) and multiple predictor variables including: `startYear, awardWins, numVotes, totalImages,`

totalVideos, totalCredits, criticReviewsTotal, awardNominationsExcludeWins, numRegions, userReviewsTotal, countryOfOrigin, genres, and titleType.

Table 4.3: Multivariate regression.

Target	Predictors	Model	R^2	MSE	MAE
averageRating, runtimeMinutes	startYear, awardWins, numVotes, totalImages, totalVideos, totalCredits, criticReviewsTotal, awardNominationsExcludeWins, numRegions, userReviewsTotal, countryOfOrigin, genres, titleType	Linear	0.031	841.81	15.27
		Lasso	0.045	816.43	15.27
		Ridge	0.031	841.72	15.27
		Decision Tree	-0.205	1264.0	19.74
		kNN	-0.201	1140.2	18.41

Notes: Best parameters of Decision Tree: {'max_depth': 5, 'min_samples_leaf': 4, 'min_samples_split': 2}. Best parameters of kNN: {'metric': 'cityblock', 'n_neighbors': 10, 'weights': 'distance'}.

- Linear Models (Linear, Ridge, Lasso): Multivariate regression slightly improved R^2 for Lasso (0.045) compared to the near-zero R^2 values of the univariate linear models. However, this improvement was negligible, with high MSE (816-842) and MAE ($\approx$ 15.27) values indicating that combining predictors did not add meaningful predictive value.
- Non-Linear Models (Decision Tree and kNN): Both the Decision Tree (R^2 = -0.205) and kNN (R^2 = -0.201) performed worse than their univariate counterparts, with negative R^2 values indicating poor predictive capability. While kNN had a slightly better MSE (1140.222 vs. 1264.026) than the Decision Tree, both models exhibited high MAE (18.411 and 19.735, respectively), reflecting inefficiency and a lack of generalizability. These results suggest that the non-linear models struggled to capture meaningful patterns in the multivariate setting, likely due to noise and overfitting.

The poor performance of multivariate regression models stems from the weak predictive power of the selected features, which do not capture enough information to explain the variation in averageRating and runtimeMinutes. Additionally, noise in the dataset reduces the performance of both linear and nonlinear methods, with kNN and Decision Tree being particularly sensitive to noisy inputs. Furthermore, the complexity of the target variables, which are influenced by subjective factors such as audience preferences, is not fully captured by the predictors. Nonlinear models such as Decision Tree and kNN also suffer from overfitting, further hindering generalization to unseen data. Despite small improvements in some cases, the results are generally insignificant due to weak correlations between variables in the dataset. This highlights the need to incorporate more informative predictors and improve data preprocessing to reduce noise.

Pattern Mining

In this section, we focus on extracting meaningful patterns and relationships from the IMDb dataset using Frequent Pattern Mining and Association Rule Mining techniques. To achieve this, we evaluate two widely used algorithms: Apriori and FP-Growth. Our objective extends beyond discovering patterns; we also aim to uncover rules that could assist in filling missing values in the runtimeMinutes feature.

Given our objective, we decided to work with the original dataset. However, since our focus includes filling patterns for runtimeMinutes, we chose to drop all rows where runtimeMinutes is missing to ensure the integrity of the discovered patterns. Other features were processed in alignment with the steps detailed in the Data Understanding section. To handle numerical attributes effectively, we carefully considered multiple discretization methods and ultimately opted for manual binning of numerical attributes into four meaningful categories. This approach was chosen because our dataset contains a significant number of outliers, which could skew the results if handled through automatic binning techniques. For example, when discretizing runtimeMinutes, we referred to typical industry standards to define four distinct categories: short_runtime (0–40 minutes), medium_runtime (40–60 minutes), long_runtime (60–90 minutes), and very_long_runtime (over 90 minutes). These bins align with common audience expectations and standard movie classifications. Similarly, when discretizing startYear, we considered the historical evolution of cinema and divided the years into four meaningful eras: early_cinema_era (1878–1919), representing the origins of film; classical_cinema_era (1920–1949), capturing Hollywood's golden age; modern_cinema_era (1950–1999), representing the rise of contemporary storytelling and technological advancements; and contemporary_cinema_era (2000–present), reflecting the digital and streaming revolu-

tion. Boolean fields such as `isAdult` and `canHaveEpisodes` were transformed into categorical labels (`adult`, `not_adult`) and (`episodic`, `non_episodic`) respectively. Other categorical variables like `genres` and `countryOfOrigin` were aggregated into fewer categories as the preprocessing done during the `Classification` phase and stored as transactional lists. Variables such as `titleType` were preserved in their original form, as they naturally align with pattern mining algorithms.

5.1 Frequent Pattern

We experimented with different minimum support values at thresholds 10, 15, 20 and minimum itemset sizes $zmin \in \{2, 3, 4, 5\}$ with both Apriori and FP Growth algorithms to observe their impact on the number of frequent patterns and meaningfulness of the discovered patterns. A clear quantitative trend emerged: decreasing the support threshold significantly increased the number of frequent itemsets, revealing more granular and rare patterns at lower thresholds. Similarly, increasing the minimum itemset size ($zmin$) led to a reduction in the number of frequent itemsets filtering out simpler associations and focusing on longer, multi-attribute patterns.

From a qualitative perspective, we observed that at higher support thresholds, such as `90%`, the frequent itemsets generated primarily consist of highly common and generalized patterns. These patterns include associations like (`low_criticReviews`, `low_userReviews`) and (`not_adult`, `low_userReviews`), which dominate the dataset due to their high frequency across many records. These patterns are reliable and statistically significant, but they represent obvious trends that are already expected, lack depth, and fail to provide meaningful insights into the nuances of the dataset. In contrast, lower thresholds (`20%`, `15%`, `10%`) progressively reveal more specific, meaningful patterns, balancing statistical strength with analytical depth.

After carefully evaluating the results, the optimal parameter combination was determined to be `support = 10%` and `zmin` = 5. This configuration balances statistical reliability with interpretative depth, generating frequent itemsets that are detailed, multi-dimensional, and meaningful. Moreover, this choice aligns with our primary goal of uncovering meaningful patterns that can guide the imputation of missing values in the `runtimeMinutes` feature. At this threshold, frequent itemsets consistently include associations involving `runtimeMinutes` categories (`short_runtime`, `medium_runtime`, `long_runtime`, `very_long_runtime`), revealing insightful relationships with other attributes.

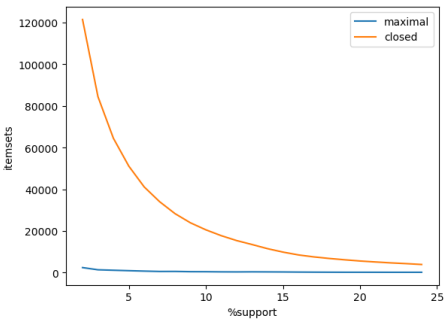


Figure 5.1: Support vs number of itemsets

The graph in Figure 5.1 reinforces our observations, illustrates the relationship between support thresholds (%) and the number of frequent itemsets, categorized into maximal and closed. As the support threshold decreases, the number of closed itemsets rises sharply, indicating that lower support values capture more granular and specific patterns. In contrast, the number of maximal itemsets remains relatively stable across all support thresholds, suggesting that only a subset of patterns represents the most significant associations. The widening gap between closed and maximal itemsets at lower thresholds highlights the increasing presence of more specific, detailed patterns within the dataset. This trend emphasizes the trade-off between statistical generalization at higher supports and pattern granularity at lower supports, with lower thresholds revealing deeper insights into the dataset's structure.

In conclusion, our analysis comparing Apriori and FP-Growth across varying support thresholds (10%, 15%, 20%) and minimum itemset sizes ($zmin \in \{2, 3, 4, 5\}$) revealed clear distinctions in their performance, scalability, and suitability for our dataset objectives. Apriori, while effective at higher support thresholds, struggled with computational efficiency and scalability at lower thresholds. In contrast, FP-Growth demonstrated superior efficiency, handling lower support thresholds with ease and consistently generating a larger set of multi-dimensional, interpretable patterns.

5.2 Association Rules

In this analysis, we observed rules generated at different confidence thresholds 70%, 60%, 50% for both Apriori and FP-Growth algorithm.

For the Apriori algorithm, the number of rules increased significantly as the confidence threshold decreased. At a 70% confidence threshold, the algorithm generated 287,580 rules. At the threshold of 60%, the number of rules rose to 327,049. At a 50% confidence threshold, the number of rules surged to 413,594. To enhance the quality and interpretability of the discovered patterns, we filtered out rules with a lift value below 2, ensuring that only the strongest and most meaningful rules were retained. Additionally, the number of consequences increased as the threshold decreased, with 9 distinct consequences identified at 50% confidence: ['short', 'short_runtime', 'medium_runtime', 'tvEpisode', 'long_runtime', 'movie', 'regional', "['Other Countries']", 'very_long_runtime'].

In contrast, for the FP-Growth algorithm, the number of rules remained constant across all confidence thresholds at 413,594 rules and 16,704 rules after filtering lift < 2. This consistency stems from FP-Growth's tree-based structure, which efficiently generates all possible frequent itemsets in a single database scan, irrespective of the confidence threshold. The filtering based on confidence happens during the rule extraction phase, and as a result, lowering or increasing the confidence threshold does not impact the overall number of rules generated. Interestingly, the number and nature of consequences in FP-Growth also remained consistent across thresholds, with the same 9 consequences as those observed in Apriori at 50% confidence. After evaluating both algorithms, we selected a confidence threshold of 50% because it aligns with our analytical goals.

One of our objectives in the Frequent Pattern Mining and Association Rule Mining analysis was to uncover meaningful association rules that could aid in imputing missing values for the runtimeMinutes feature. The analysis revealed distinct patterns across different runtime categories (see Table 5.1). Short runtimes, with the highest number of rules (1,437), are strongly associated with the short category and are typically linked to attributes such as local, small_scale_credits, very_low_images, low_votes, non_episodic, low_criticReviews, low_userReviews, and not_adult. In contrast, medium runtimes, represented by 410 rules, predominantly align with the tvEpisode category and often co-occur with features such as medium_scale_credits, high_rating, no_nominations, non_episodic, no_awards, low_userReviews, and not_adult. Similarly, long runtimes, with 378 rules, are closely tied to the movie category and are characterized by antecedents including medium_rating, small_scale_credits, no_videos, low_votes, no_nominations, non_episodic, and low_criticReviews. These findings offer clear and actionable guidelines for imputing missing runtimeMinutes values based on categorical associations. Specifically, entries categorized as short can be imputed with a short runtime, those labeled as tvEpisode with a medium runtime, and those tagged as movie with a long runtime. These results reaffirm the correctness of our initial choice in the Data Understanding phase, where we used the median runtimeMinutes for each titleType category as an imputation strategy.

Moreover, we also observed consequences such as tvEpisode, movie, and short categories revealed some insights (see Table 5.1). For tvEpisode, rules indicate strong associations with features such as medium_scale_credits, high_rating, local, and not_adult. These attributes suggest that TV episodes are often medium-scale productions, highly rated, localized, and generally suitable for a non-adult audience. This insight highlights why TV episodes tend to secure higher ratings and emphasizes the importance of focusing on these attributes for future investments in episodic content. In the case of short, the rules reveal significant associations with short_runtime, contemporary_cinema_era, small_scale_credits, and not_adult. These patterns suggest that short films are often produced with small budgets, targeted at contemporary audiences, and typically non-adult content. This knowledge offers valuable input for decision-making on low-budget short films, emphasizing their viability and alignment with specific audience preferences. For movie, the analysis uncovered associations with medium_rating, non_episodic, low_criticReviews, low_userReviews, and not_adult. These attributes imply that movies are typically medium-rated, non-episodic productions with moderate audience engagement and lower critical reviews. These associations not only provide insights into runtime behavior but also guide strategic decision-making in film production and investment planning.

Table 5.1: Association rules

Consequent	**Antecedent**	**support** -	**support** %	**confidence** -	**lift** -
long_runtime	(movie, medium_rating, small_scale_credits, no_videos, low_votes, no_nominations, non_episodic,low_criticReviews)	666	5.75	0.52	2.39
	(movie, medium_rating, small_scale_credits, no_videos, low_votes, no_nominations)	666	5.75	0.52	2.39
medium_runtime	(tvEpisode, medium_scale_credits, high_rating, no_nominations, no_awards, low_userReviews, not_adult)	654	5.65	0.57	2.76
	(tvEpisode, contemporary_cinema_era, no_videos, no_awards, low_criticReviews, low_userReviews, not_adult)	680	5.81	0.50	2.57
short_runtime	(short, local, small_scale_credits, very_low_images, low_votes, no_nominations, low_userReviews, not_adult)	1190	10.28	1.00	2.77
	(short, local, small_scale_credits, very_low_images, low_votes, no_nominations, low_userReviews)	1192	10.28	1.00	2.77
tvEpisode	(medium_runtime, high_rating, very_low_images, low_criticReviews, low_userReviews, not_adult)	811	7.00	0.70	2.49
	(medium_runtime, high_rating, very_low_images, low_criticReviews, low_userReviews)	811	7.00	0.69	2.48
movie	(long_runtime, medium_rating, no_videos, low_votes, non_episodic, low_criticReviews, low_userReviews)	932	8.05	0.78	2.10
	(long_runtime, medium_rating, no_videos, low_votes, non_episodic, low_criticReviews)	933	8.06	0.78	2.10
short	(short_runtime, contemporary_cinema_era, small_scale_credits, low_votes, non_episodic, low_userReviews, not_adult)	881	7.61	0.71	4.36
	(short_runtime, contemporary_cinema_era, small_scale_credits, low_votes, non_episodic, low_criticReviews, low_userReviews, not_adult)	880	7.60	0.71	4.36